

LangChain 《State of Agent Engineering》本质上是一份 **AI Agent 工程化现状调研报告**：它不是讲某一个 Agent 技术细节，而是回答一个更现实的问题——**企业现在到底怎么用 Agent、卡在哪里、哪些工程能力已经变成必需品**。报告基于 2025 年 11 月 18 日至 12 月 2 日的公开调研，共 1340 份回复。

AI Agent 已经从“概念验证”进入“生产部署”阶段，但真正的瓶颈不再是模型能力或成本，而是质量、延迟、可观测性、评估体系、安全合规和上下文工程。

换句话说，今天做 Agent，重点已经不是“能不能让 LLM 调工具”，而是：

如何把一个不稳定、非确定性的 LLM，工程化成一个可靠、可调试、可评估、可上线、可规模化运行的系统。

1. 什么是 Agent Engineering?

报告给出的定义是：

Agent Engineering 是把 LLM 组织成可靠系统的迭代过程。

这句话很关键。它意味着 Agent 工程不是一次性写 Prompt，也不是简单接几个 API，而是一个持续迭代的系统工程。

可以通俗理解为：

普通 LLM 应用像是“让模型回答问题”；

Agent 系统则像是“让模型承担一段工作流程”。

例如：

用户问一句话，普通 LLM 直接生成回答；

Agent 可能要经历：

1. 理解任务；
2. 决定是否需要调用工具；
3. 检索资料；
4. 读取数据库；
5. 执行代码或调用 API；
6. 判断结果是否足够；
7. 必要时继续迭代；
8. 最后生成结构化输出。

所以 Agent 最大的工程难点在于：它不是单步生成，而是多步决策系统。

这也解释了为什么报告反复强调 observability、evals、latency、quality、context engineering。

2. Agent 已经进入生产阶段，尤其是大企业

报告显示，57.3% 的受访组织已经有 Agent 在生产环境运行，另有 30.4% 正在积极开发并计划上线。相比上一年 51% 的生产部署比例，Agent 的生产化趋势明显增强。

更值得注意的是，大企业采用速度更快：

员工数超过 1 万人的组织中，67% 已经有 Agent 在生产环境运行，24% 正在开发并计划上线。

这说明 Agent 已经不只是创业公司或技术团队的玩具，而是逐渐进入大型组织的正式生产系统。

通俗解释：

以前大家问的是：

Agent 到底有没有用？

现在企业问的是：

Agent 怎么上线？怎么稳定？怎么监控？怎么控制风险？怎么规模化？

这正是从“demo 阶段”到“工程化阶段”的典型变化。

3. 当前最主流的 Agent 应用场景

报告中最主要的 Agent 用例有三类：

第一是 客户服务，占比 26.5%。

第二是 研究与数据分析，占比 24.4%。

第三是 内部效率提升 / 工作流自动化，占比约 18%。

这几个场景很有代表性。

3.1 客户服务：最容易规模化，也最容易暴露问题

客服场景天然适合 Agent，因为它有大量重复性问题，也有明确的业务流程，比如查订单、查政策、改地址、生成回复。

但它也最考验稳定性，因为 Agent 是直接面对客户的。一旦幻觉、语气不当、答错政策，就会带来真实业务风险。

3.2 研究与数据分析：Agent 的优势场景

报告强调，Agent 很适合处理大量信息、跨来源综合、辅助知识密集型任务。

这和你目前学习的 RAG、金融研究 Agent、研报问答系统高度相关。

比如一个金融研究 Agent 可以做：

1. 读取市场数据；
2. 识别异常指标；
3. 检索相关新闻；
4. 综合可能原因；
5. 生成周报卡片；
6. 标注置信度和引用来源。

这类任务的核心不是“单次问答”，而是 **数据 + 工具 + 推理 + 报告生成** 的完整链条。

3.3 内部效率提升：大企业尤其关注

在 1 万人以上组织中，内部生产力是最主要用例，占比 26.8%。

这也很好理解。大企业有大量内部流程、知识库、审批、报表、客服、合规、IT 支持等场景。Agent 不一定先替代核心业务，而是先提升内部流程效率。

4. 最大生产瓶颈：质量，而不是成本

报告最重要的结论之一是：

质量是 Agent 上生产的最大阻碍。

约三分之一受访者将质量列为主要 blocker。这里的质量包括：

1. 准确性；
2. 相关性；
3. 一致性；
4. 语气是否合适；
5. 是否遵守品牌、政策、合规要求。

这对理解 Agent 非常关键。

很多初学者会以为 Agent 最大问题是“模型不够聪明”或者“API 太贵”。但报告显示，真正卡住生产的，是：

Agent 能不能稳定地产生符合业务标准的结果。

例如一个客服 Agent，回答 100 次里 95 次正确，但 5 次胡说八道，可能就不能上线。

一个金融研究 Agent，90% 的归因合理，但 10% 把相关性误写成因果关系，也会有很大风险。

所以 Agent 工程的核心不是追求“看起来很聪明”，而是追求：

可控、稳定、可解释、可回溯。

5. 第二大挑战：延迟

报告显示，**延迟 / 响应时间**是第二大生产挑战，占比约 20%。

这也非常符合 Agent 的本质。

普通 LLM 问答可能只调用一次模型。

但 Agent 可能需要：

1. 调一次模型做规划；
2. 调工具检索；
3. 再调模型分析；
4. 再调数据库；
5. 再调模型生成；
6. 甚至循环多轮。

所以 Agent 越复杂，越可能慢。

这里存在一个典型 trade-off：

更强的 Agent 往往需要更多步骤，但更多步骤会带来更高延迟。

例如：

简单客服 Bot：快，但能力弱。

复杂 Agent：会查资料、调系统、判断政策，但响应慢。

所以生产系统里常见的做法是：

1. 简单问题走快速路径；
2. 复杂问题才进入多步 Agent；

3. 高风险任务引入人工确认；
 4. 对不同任务使用不同模型；
 5. 对工具调用、检索、生成做缓存和并发优化。
-

6. 安全合规在大企业中特别重要

报告提到，在 2000 人以上企业中，安全成为第二大 concern，占比 24.9%，超过了延迟。

这点很重要，尤其对金融、医疗、政府、大型企业。

Agent 的安全问题不是单纯“模型会不会胡说”，还包括：

1. 能不能访问不该访问的数据；
2. 会不会泄露客户隐私；
3. 工具调用权限是否过大；
4. 是否会执行错误操作；
5. 是否符合审计要求；
6. 是否能留下操作记录；
7. 是否支持权限隔离。

通俗讲，普通 Chatbot 最多“说错话”；

Agent 如果接入工具和系统，可能会“做错事”。

比如：

误发邮件、误删数据、错误下单、错误审批、错误调用内部接口。

所以 Agent 的生产化必须有权限控制、审计日志、人工确认、沙箱环境和回滚机制。

7. 可观测性已经成为 Agent 的标配能力

报告中一个非常关键的数据是：

89% 的组织已经为 Agent 实施了某种形式的 observability，其中 62% 可以追踪具体 Agent 步骤和工具调用。

对于已经有 Agent 在生产环境的组织，比例更高：**94% 有 observability，71.5% 有完整 tracing 能力。**

这说明：没有可观测性，就很难做生产级 Agent。

Agent 的可观测性至少要看这些东西：

1. 用户输入是什么；
2. Agent 的中间推理 / 决策路径是什么；
3. 调用了哪些工具；
4. 工具输入输出是什么；
5. 检索到了哪些文档；
6. 最终答案基于哪些证据；
7. 哪一步失败了；
8. 失败是模型问题、检索问题、工具问题，还是上下文问题；
9. 延迟和 token 成本分别消耗在哪些环节。

通俗理解：

普通软件出 bug，你可以看日志。

Agent 出 bug，如果没有 tracing，你甚至不知道它为什么错。

所以 LangSmith、OpenTelemetry、日志链路追踪、tool call tracing 这类能力会越来越重要。

8. Evals 还在追赶，但越来越重要

报告指出，52.4% 的组织会在测试集上做离线评估，37.3% 会基于生产数据做在线评估。已经有 Agent 上生产的组织中，在线评估比例更高，达到 44.8%。

这里可以拆成两种 eval：

8.1 Offline Eval：上线前测试

就是准备一批标准测试问题，看 Agent 表现如何。

例如研报问答系统可以准备：

1. “某公司 2023 年收入是多少？”
2. “请总结这篇研报的核心观点。”
3. “这个结论来自第几页？”
4. “如果材料没有提到，请回答不知道。”
5. “不要编造引用。”

然后检查 Agent 是否答对、是否引用准确、是否遵守格式。

8.2 Online Eval：上线后监控

就是 Agent 真实面对用户后，继续评估它的表现。

因为测试集永远覆盖不了真实世界。用户会问奇怪的问题，数据会变化，业务规则会更新，工具也可能失败。

所以生产级 Agent 需要持续监控：

1. 用户满意度；
2. 幻觉率；
3. 工具调用失败率；
4. 引用错误率；
5. 人工接管率；
6. 任务完成率；
7. 响应延迟；
8. 成本。

这 and 传统软件测试不同。传统软件更多是确定性逻辑，而 Agent 是非确定性的，因此 eval 不是一次性工作，而是持续工程。

9. 人工评审和 LLM-as-Judge 并存

报告显示，在评估方式上：

人工评审 / 标注占 59.8%，LLM-as-judge 占 53.3%。传统机器学习指标如 ROUGE、BLEU 使用较少。

这说明 Agent 评估更依赖语义判断，而不是简单字符串匹配。

比如同一个问题可以有多个合理回答，不能只看字面重合度。

为什么 ROUGE、BLEU 不适合 Agent?

因为它们主要衡量文本和参考答案的重合度，更适合翻译、摘要等有参考答案的任务。

但 Agent 任务往往是开放式的：

用户问：“请分析这周市场异动原因。”

可能有多个合理答案。只要事实正确、逻辑清晰、证据充分，就都可以接受。

LLM-as-Judge 的价值

LLM-as-Judge 可以规模化评估：

1. 答案是否相关；
2. 是否遵守格式；
3. 是否有幻觉；
4. 是否引用了证据；
5. 语气是否符合要求；
6. 是否完成任务。

但高风险场景仍然需要人工评审，尤其是金融、医疗、法律、客服投诉等。

比较成熟的方式是：

自动评估负责广覆盖，人工评审负责高风险和深度判断。

10. 多模型使用成为常态，Fine-tuning 反而不是主流

报告显示，OpenAI GPT 模型采用率最高，超过三分之二组织使用；但超过四分之三组织会使用多个模型。Gemini、Claude、开源模型也都有显著采用。

这说明企业并不想完全绑定一个模型供应商，而是会根据任务做模型路由：

1. 高复杂推理任务用强模型；
2. 简单分类任务用便宜模型；
3. 低延迟任务用快模型；
4. 敏感数据任务用私有化或开源模型；
5. 长上下文任务用长上下文模型；
6. 代码任务用代码能力强的模型。

报告还提到，**57% 的组织没有 fine-tune 模型，而是==依赖基础模型 + prompt engineering + RAG。==**

这点非常重要。

很多人会误以为企业做 AI 应用一定要微调模型。但现实中，Agent 场景通常优先考虑：

1. Prompt；
2. RAG；
3. 工具调用；
4. workflow编排；
5. 上下文工程；
6. 评估与监控；

7. 多模型路由。

Fine-tuning 更适合一些专门任务，比如固定风格、专门分类、特定领域格式输出等，但不是 Agent 工程的默认起点。

11. 日常使用最多的 Agent：代码 Agent 和研究 Agent

报告的开放回答显示，日常使用最多的是代码 Agent，比如：

1. Claude Code；
2. Cursor；
3. GitHub Copilot；
4. Amazon Q；
5. Windsurf；
6. Antigravity。

第二类是研究 / deep research Agent，比如 ChatGPT、Claude、Gemini、Perplexity 等。第三类是基于 LangChain / LangGraph 搭建的自定义 Agent，用于 QA 测试、知识库搜索、SQL / Text-to-SQL、需求规划、客服和工作流自动化。

这个结论也很符合当前行业趋势：

Agent 最先爆发的两个场景，一个是写代码，一个是做研究。

因为这两个场景都有几个共同点：

1. 信息量大；
 2. 上下文复杂；
 3. 任务可以拆步骤；
 4. 用户愿意和 Agent 多轮交互；
 5. 结果可以由人类校验；
 6. 工具调用价值明显。
-

12. 这份报告对理解 Agent 的关键启发

12.1 Agent 的核心壁垒不是“会调工具”，而是“可靠完成任务”

很多教程会把 Agent 讲成：

LLM + Tool Calling + Memory + Planning。

这只是结构层面的定义。

真正上生产时，企业关心的是：

1. 是否稳定；
2. 是否准确；
3. 是否可追溯；
4. 是否可评估；
5. 是否安全；
6. 是否可控；
7. 是否能和现有系统集成；
8. 是否能承担业务责任。

我如何设计了数据层、工具层、决策层、评估层、日志追踪和降级机制，使 Agent 能在不确定环境下可靠运行。

12.2 未来 Agent 工程师的核心能力栈

从报告看，一个合格的 Agent Engineer 至少要懂六类能力：

第一，**任务拆解能力**：把业务问题拆成可执行流程。

第二，**工具设计能力**：定义工具 schema、输入输出、权限边界。

第三，**上下文工程能力**：决定给模型什么信息、不给什么信息、如何压缩和组织上下文。

第四，**评估能力**：设计 offline eval、online eval、LLM-as-judge、人审流程。

第五，**可观测性能力**：trace 每一步决策、工具调用、失败原因、成本和延迟。

第六，**生产工程能力**：部署、权限、安全、监控、回滚、降级、成本控制。

这其实已经不是单纯 NLP 工程，而是：

LLM 应用架构 + 后端工程 + 数据工程 + 产品判断 + 风险控制。

12.3 Agent 最难的是“非确定性系统的工程化”

传统软件是：

输入 A → 程序逻辑固定 → 输出 B。

Agent 更像是：

输入 A → 模型判断 → 工具调用 → 中间状态变化 → 再判断 → 可能输出 B / C / D。

它的行为具有不确定性。

所以 Agent 工程不是追求完全消灭不确定性，而是要通过工程手段管理不确定性：

1. 用 schema 限制输出；
2. 用工具权限限制动作；
3. 用检索证据约束回答；
4. 用评估集发现回归；
5. 用 tracing 定位错误；
6. 用人工审核兜底高风险任务；
7. 用降级机制处理失败；
8. 用规则引擎承接确定性逻辑。

13. 用一个通俗类比理解 Agent 工程

可以把 Agent 想象成一个新入职的分析师。

普通 LLM 像是一个“会说话的顾问”，你问它，它回答。

Agent 更像是一个“可以干活的初级员工”：

你给它一个任务，它会自己查资料、整理数据、调用系统、写报告。

但问题是，这个员工：

1. 有时会误解任务；
2. 有时会查错资料；
3. 有时会过度自信；
4. 有时会跳步骤；
5. 有时会编理由；
6. 有时不知道自己错了。

所以企业不能直接让它裸奔上岗，而要给它配：

1. SOP；
2. 权限边界；
3. 任务清单；
4. 审核机制；
5. 工作日志；
6. 绩效评估；

7. 异常上报；
8. 人工主管。

这就是 Agent Engineering。

14. 对金融场景的特别启示

这份报告对金融行业尤其有参考意义。

金融 Agent 很可能优先落在这些场景：

1. 研报问答；
2. 市场异动归因；
3. 客户服务；
4. 内部知识库搜索；
5. 投研材料整理；
6. KPI / 经营数据分析；
7. 合规问答；
8. 尽调材料摘要；
9. 路演纪要生成；
10. 数据口径核对。

但金融场景的风险也更高：

1. 幻觉风险；
2. 数据时效风险；
3. 引用错误风险；
4. 权限泄露风险；
5. 投资建议边界风险；
6. 合规措辞风险；
7. 归因过度确定风险；
8. 内外部数据混用风险。

所以金融 Agent 的设计重点不应该是“让模型自由发挥”，而应该是：

确定性数据计算 + 可追溯检索 + 受控推理 + 结构化输出 + 引用溯源 + 人工审核 + 降级机制。

这也和你之前做的 Market Insight Agent 思路高度一致：Data 层负责确定性计算，Agent 层负责解释和决策，Report 层负责稳定渲染。

15. 这份报告可以沉淀成一个 Agent 工程判断框架

以后你判断一个 Agent 项目是否靠谱，可以问八个问题：

1. **任务是否适合 Agent?**

是单步问答，还是多步任务？

2. **Agent 需要哪些工具?**

搜索、数据库、代码执行、内部 API、文件读取？

3. **工具权限如何控制?**

能读什么？能写什么？哪些操作需要人工确认？

4. **上下文如何组织?**

哪些信息进入 prompt？哪些通过 RAG 检索？如何避免上下文过长？

5. **质量如何评估?**

有没有测试集？有没有指标？有没有人工 review？

6. **线上如何监控?**

是否能看到每一步 tool call、延迟、失败、成本、用户反馈？

7. **失败如何降级?**

是拒答、转人工、返回部分结果，还是进入保守模式？

8. **是否能规模化?**

成本、延迟、并发、安全、合规是否可控？

这八个问题，就是从“做 demo”走向“做生产系统”的分水岭。